

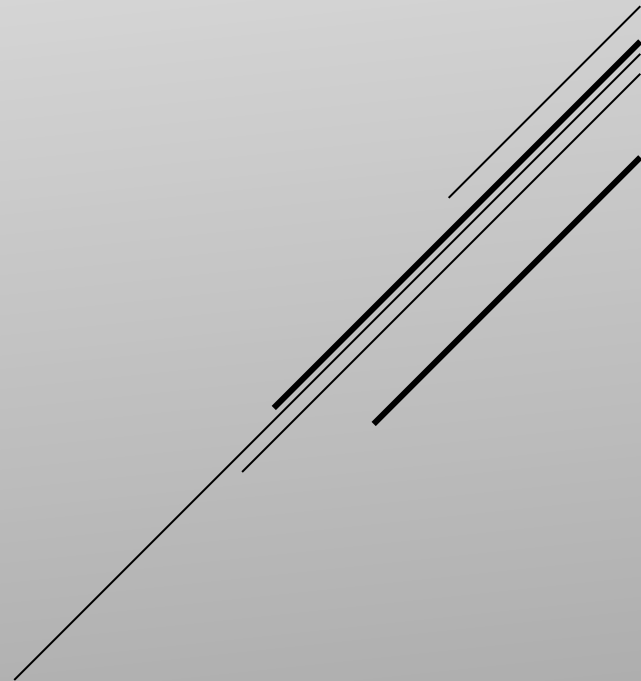
PRACTICA 4: DETECCIÓN DE NEUMONÍA EN BASE A RAYOS-X

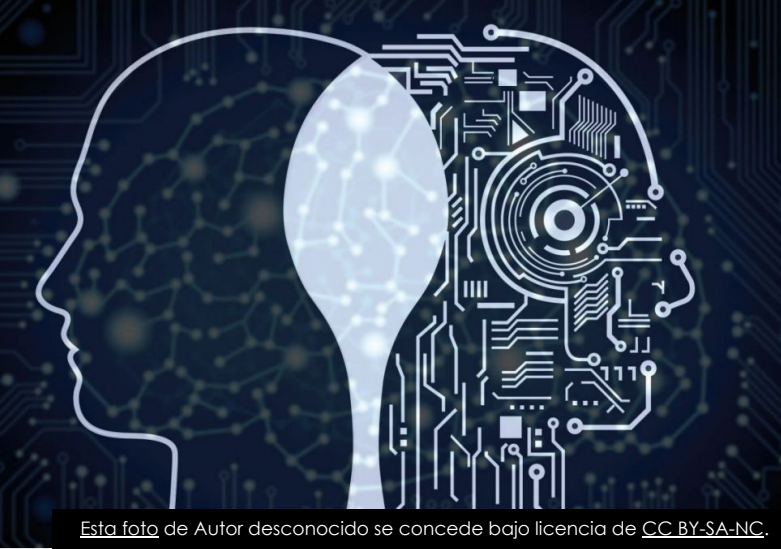
GRUPO 4

Andrés VIÑÉ SANCHEZ, BEATRIZ AEDO DÍAZ, CANDELA
ESQUINAS SÁNCHEZ, JOSÉ ANTONIO RUIZ HEREIDA

INTRODUCCIÓN

CAPÍTULO 1





Esta foto de Autor desconocido se concede bajo licencia de [CC BY-SA-NC](#).



Esta foto de Autor desconocido se concede bajo licencia de [CC BY](#).

INTELIGENCIA ARTIFICIAL EN NUESTRAS VIDAS

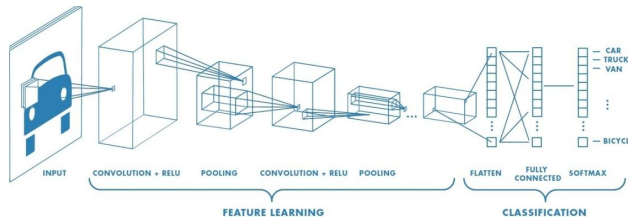
Esta foto de Autor desconocido se concede bajo licencia de [CC BY-NC-ND](#).

1.1. MOTIVACIÓN

- Pandemia del COVID-19
- Aplicación de técnicas de IA para facilitar el trabajo de los médicos



- ▶ Detección de neumonía mediante radiografías
- ▶ Aplicación de redes neuronales para este propósito



1.2. OBJETIVOS

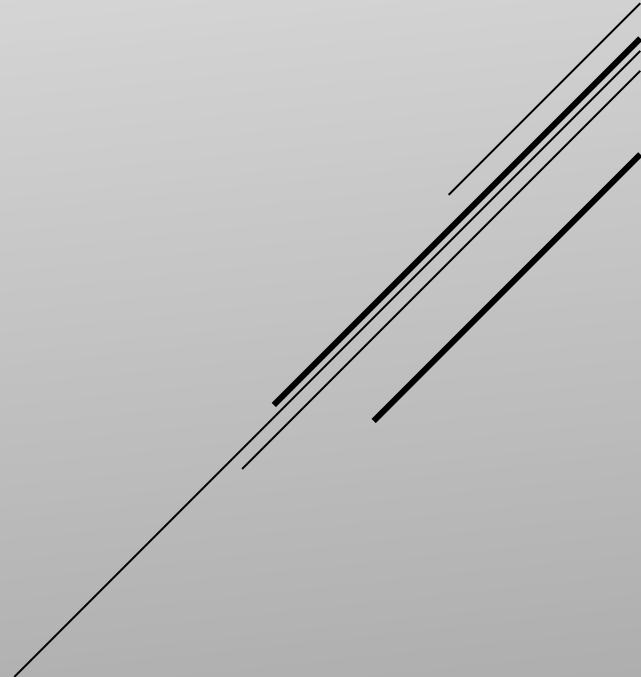
- ▶ Capítulo 1: Introducción
- ▶ Capítulo 2: Primeros Pasos
- ▶ Capítulo 3: Preprocesamiento y edición de imágenes
- ▶ Capítulo 4: Modelo
- ▶ Capítulo 5 Resultados
- ▶ Capítulo 6 Conclusión y futuros trabajos

1.3. PLAN DE TRABAJO

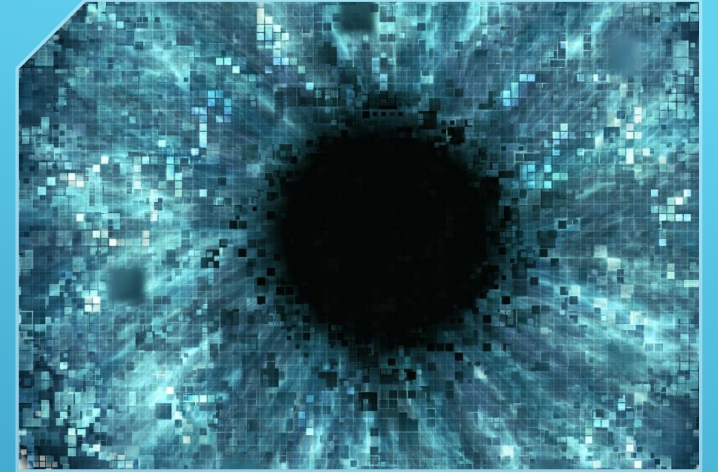


PRIMEROS PASOS

CAPÍTULO 2



- ▶ 5856 imágenes de radiografías
- ▶ Divididas en dos clases: Neumonía y Normal



kaggle



2.1. ELECCIÓN DEL DATASET

2.2. CARGA DE DATOS



Separación de las imágenes por categorías: añadiendo las etiquetas Neumonía y Normal



Separación en entrenamiento, validación y test

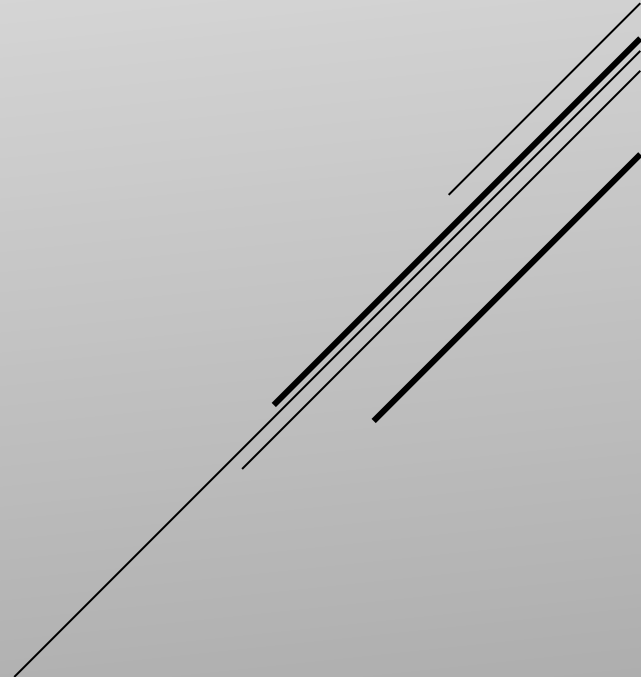


Cargar las nuevas etiquetas en un CSV



PREPROCESAMIENTO

CAPÍTULO 3



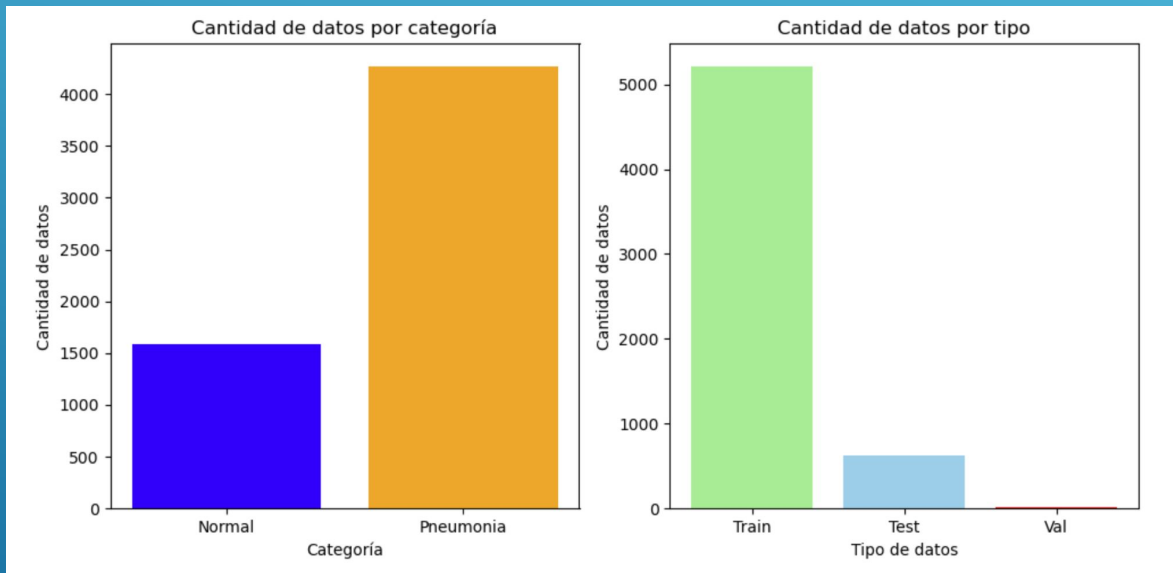
3.1. IMPORTACIÓN DE LIBRERIAS Y CARGA DE DATOS DEL CSV

- Tensorflow, sklearn, numpy, pandas...

```
# TensorFlow
import tensorflow as tf
from tensorflow.keras.models import load_model, save_model
from tensorflow.keras.applications.resnet_v2 import ResNet50V2
from tensorflow.keras.optimizers import RMSprop
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications.resnet_v2 import preprocess_input
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam

# Otros
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import glob
import os
import time
import cv2
import seaborn as sns
from Data_Paths import *
from keras.applications.resnet50 import ResNet50
```

- ▶ Desequilibrio en la cantidad de imágenes de cada clase
- ▶ Separación en función de entrenamiento validación y test



Cantidad de datos por categoría:

Normal: 1583

Pneumonia: 4273

Cantidad de datos por tipo:

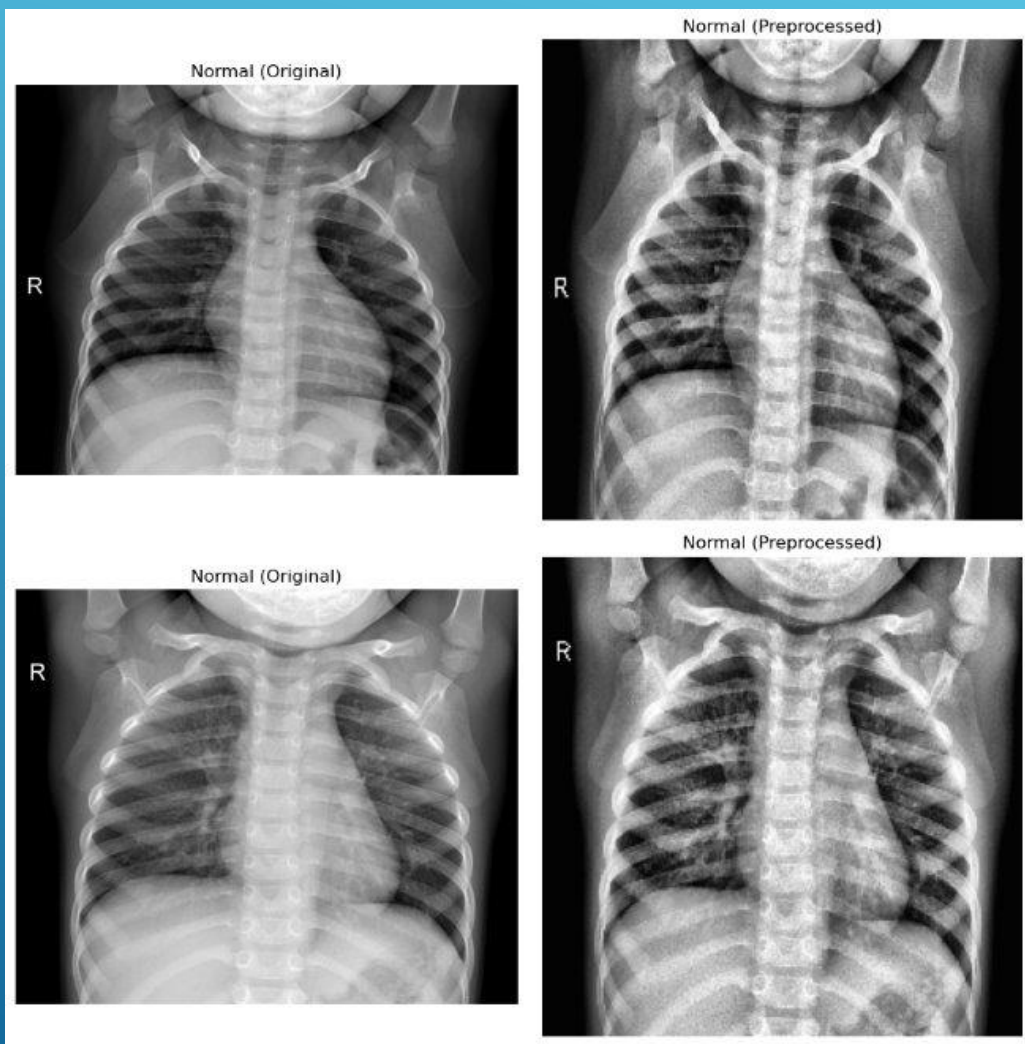
Train: 5216

Test: 624

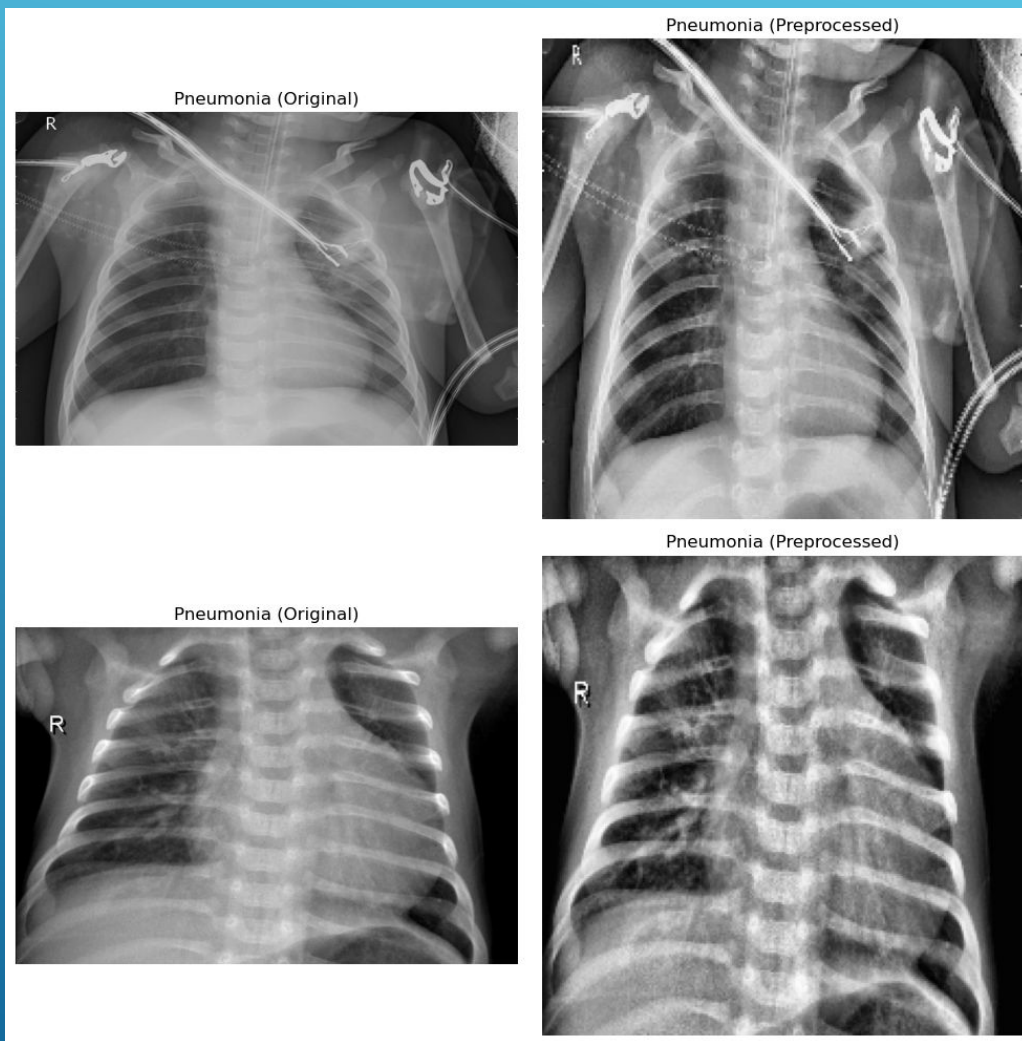
Val: 16

- ▶ Importante para el rendimiento y precisión del modelo
- ▶ Primera etapa: tamaño, niveles de grises, suavizado, contraste de las imágenes

3.2. PREPROCESADO: PRIMERA ETAPA



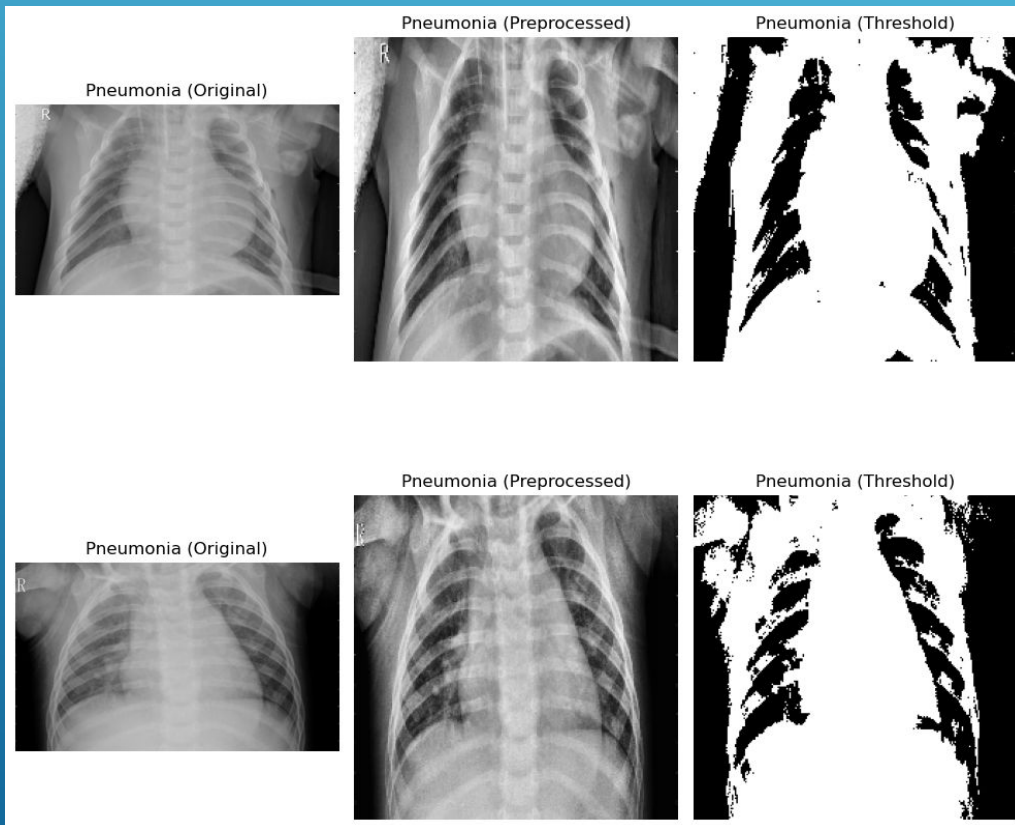
- Comparación de imágenes de categoría Normal



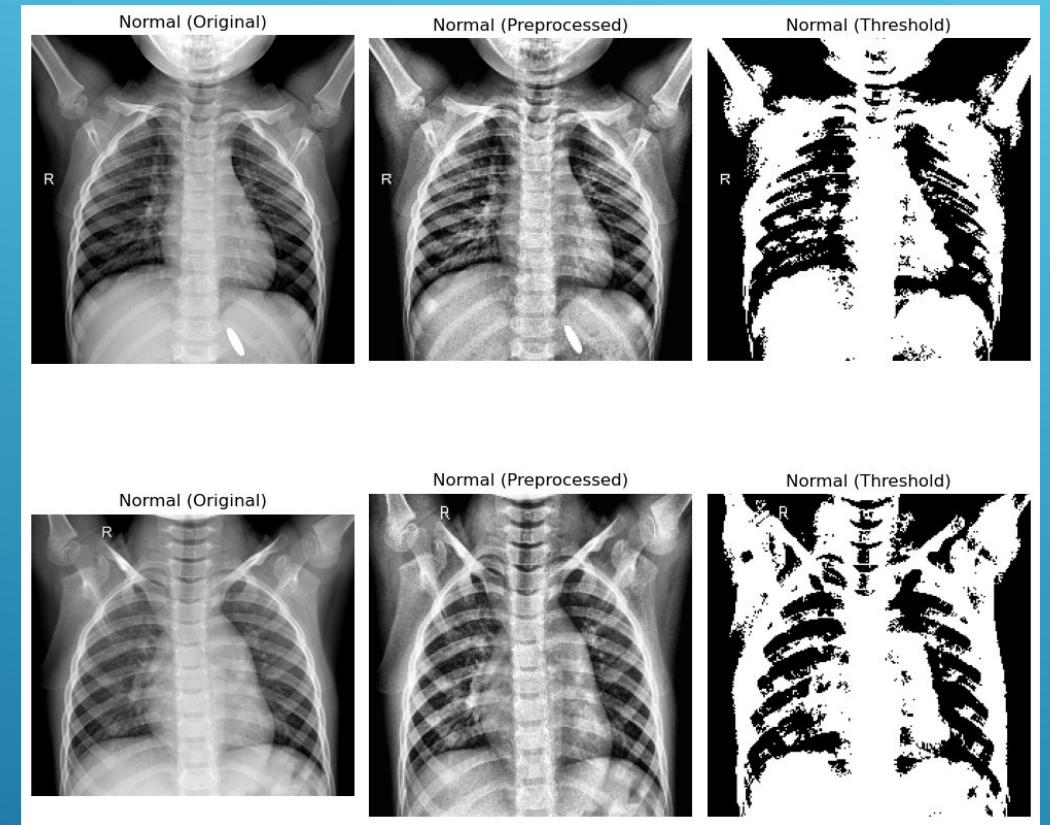
- Comparación de imágenes de categoría Neumonía

3.3. PREPROCESADO: SEGUNDA ETAPA

Comparación de las imágenes editadas con las imágenes originales



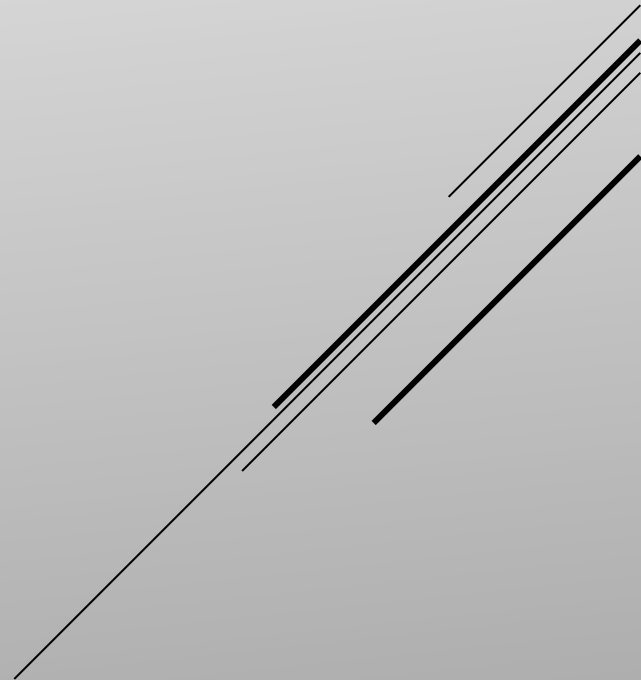
Comparación de imágenes con Neumonía con filtro y umbral



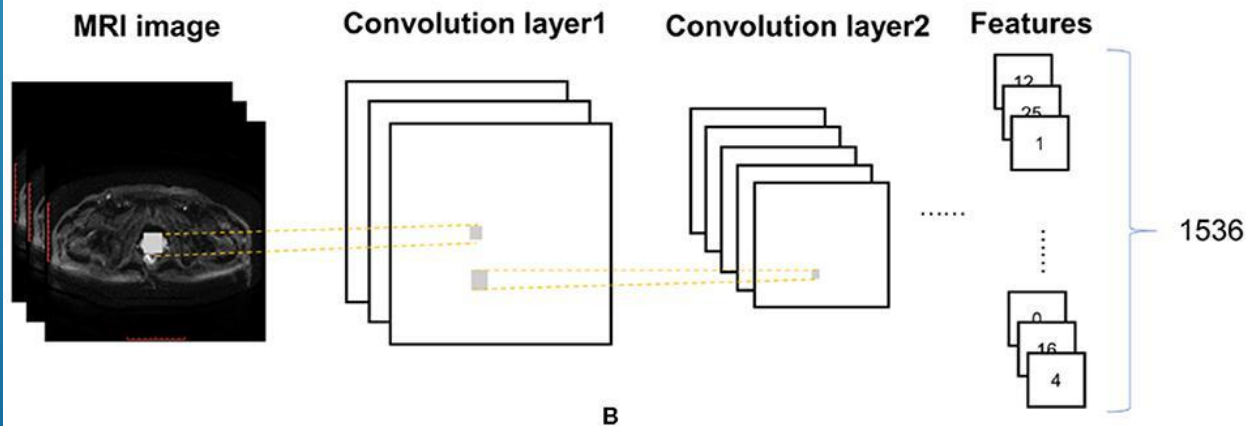
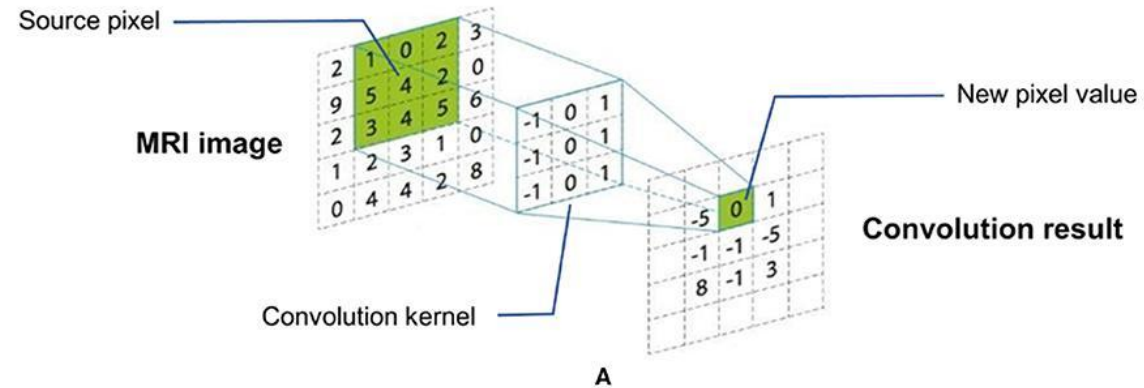
Comparación de imágenes Normales con filtro y umbral

MODELO

CAPÍTULO 4

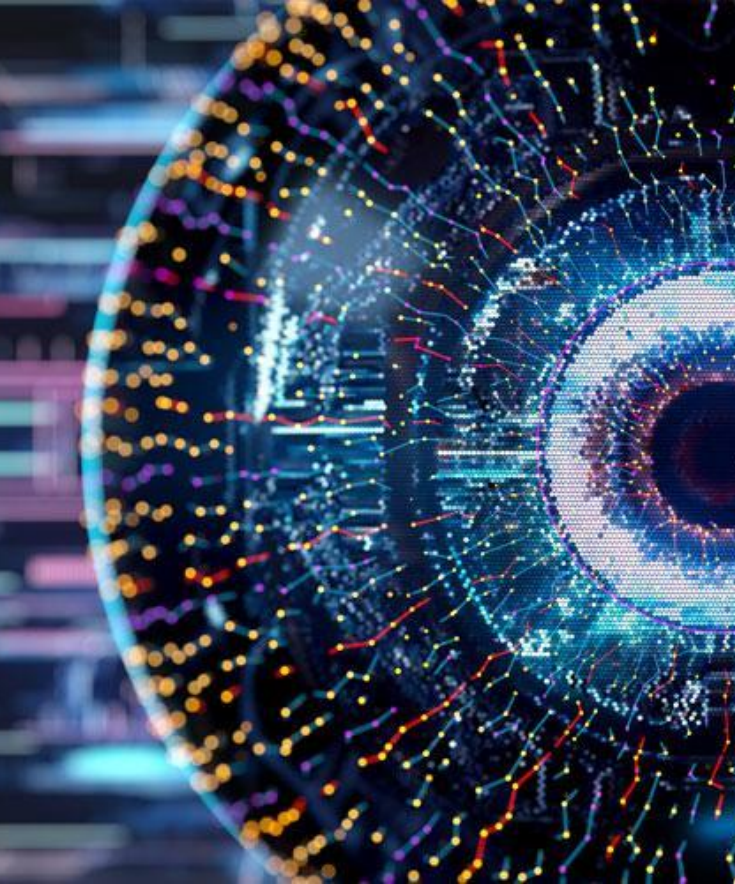


Convolution operation and extract features



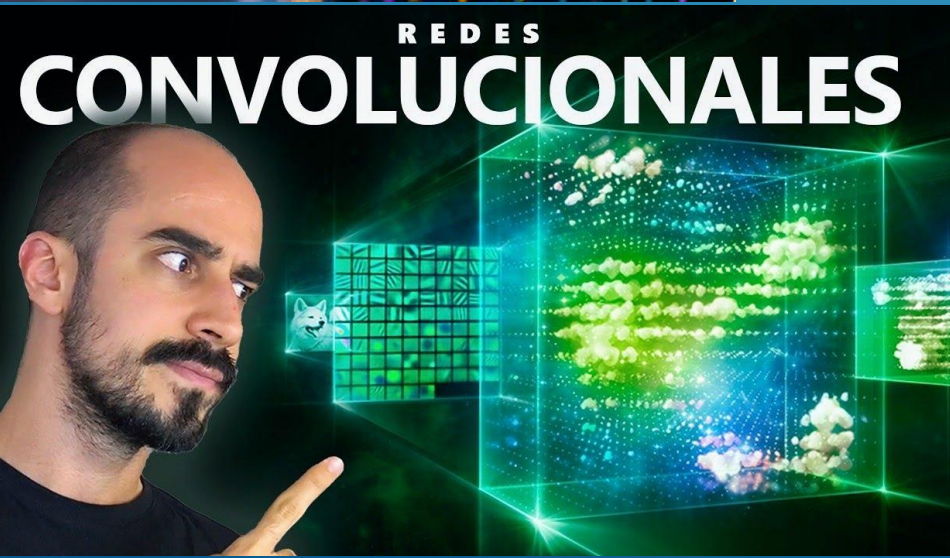
4.1. INTRODUCCIÓN

- Utilizamos un modelo pre-entrenado
- Basado en arquitectura de redes neuronales convolucionales (CNN)
- Modelo para procesamiento de imágenes



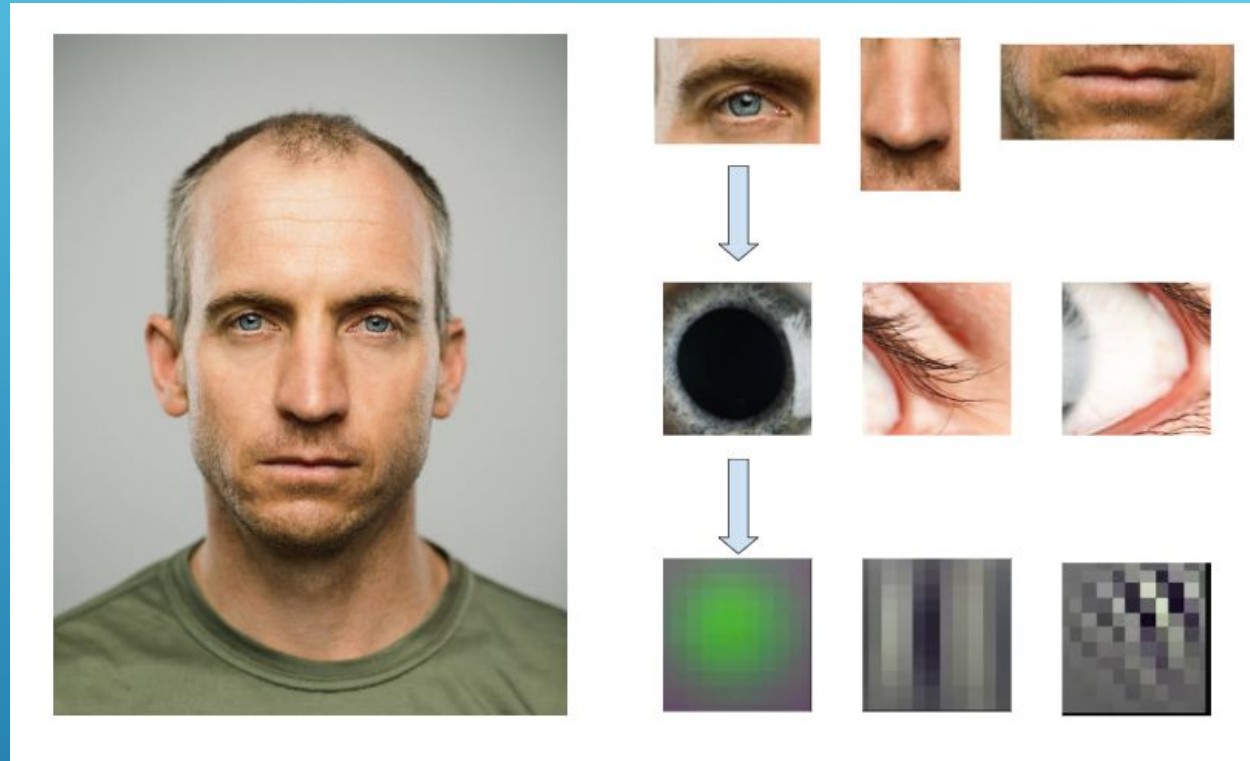
- ▶ Arquitectura de Redes Neuronales que mejor funciona para el reconocimiento de imágenes
- ▶ Ha supuesto una revolución en el campo de la visión por computador

4.2. REDES NEURONALES CONVOLUCIONALES (CNN)



4.2.1. INSPIRACIÓN BIOLÓGICA DE LAS CNN

- Basadas en el proceso de visión humano
- Procesamiento en cascada: primero se detectan los elementos más básicos y luego se estos se combinan para formar patrones más complejos



4.2.2. CONVOLUCIÓN

- Las CNN son redes neuronales con capas en las que se aplica una operación llamada convolución
- La aplicación de esta operación da como resultado imágenes denominadas mapas de características
- El primer filtro aplicará un desenfoque en la imagen
- Mientras que el segundo detecta patrones de líneas verticales

Filtro

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

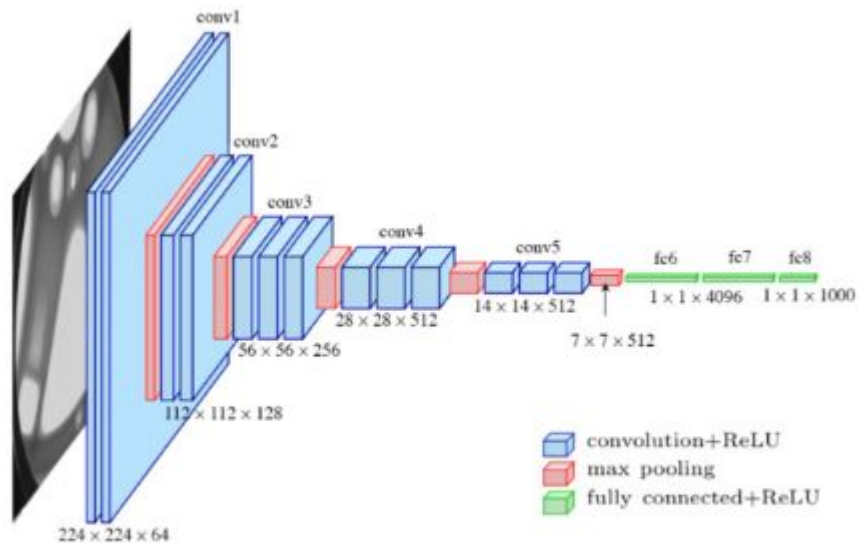


Filtro

-1	0	1
-1	0	1
-1	0	1



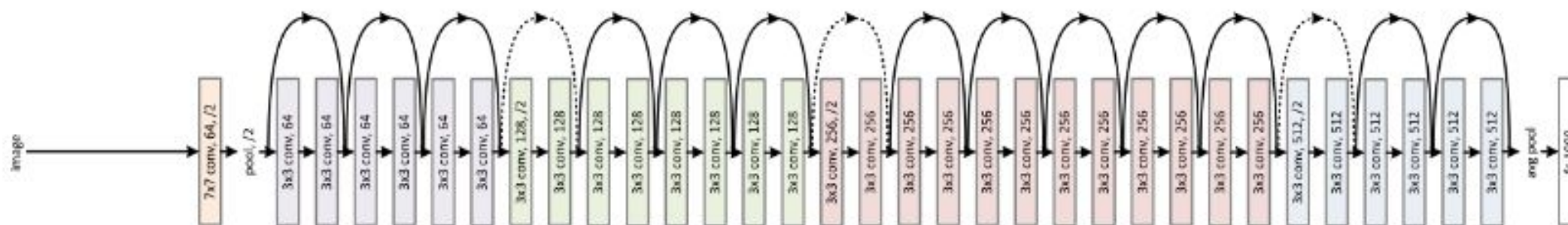
- La aplicación repetida de estos filtros puede hacer que aparezcan patrones más complejos que simples líneas verticales
- Al igual que la visión humana las CNN procesan las imágenes en cascada
- Representación de las CNN como un embudo



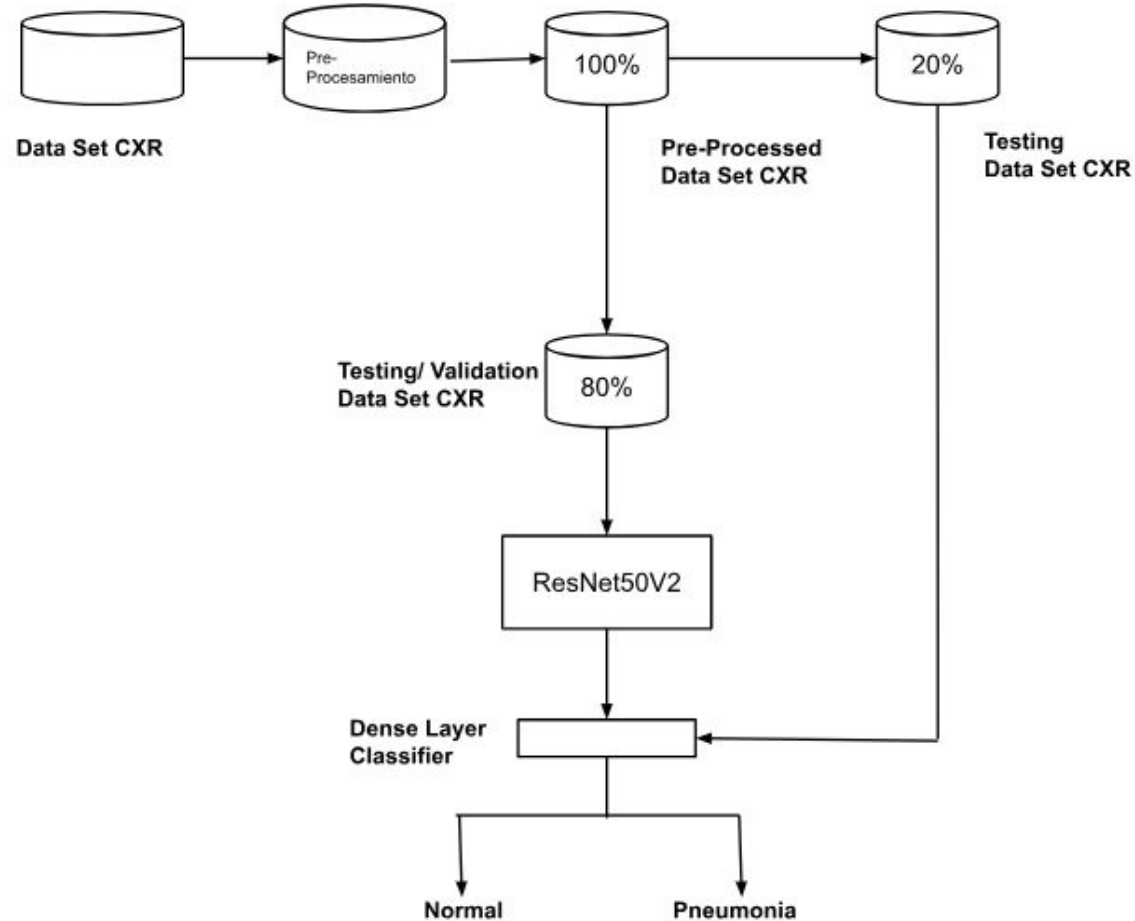
"Diagrama de una CNN meramente explicativo (Independiente del modelo ResNet50V2)"

4.3. RESNET50V2

- ▶ De los modelos más usados en visión por computador
- ▶ Versión simplificada del modelo ResNetV2 (50 capas vs. 125)
- ▶ Cuenta con capas convolucionales, de normalización y utiliza la función ReLU como función de activación
- ▶ Arquitectura de bloques residuales (Residual Net). Implementa conexiones entre capas llamadas skip connections



"Diagrama del Modelo ResNetV2 (Versión de mayor tamaño pero estructura similar)"

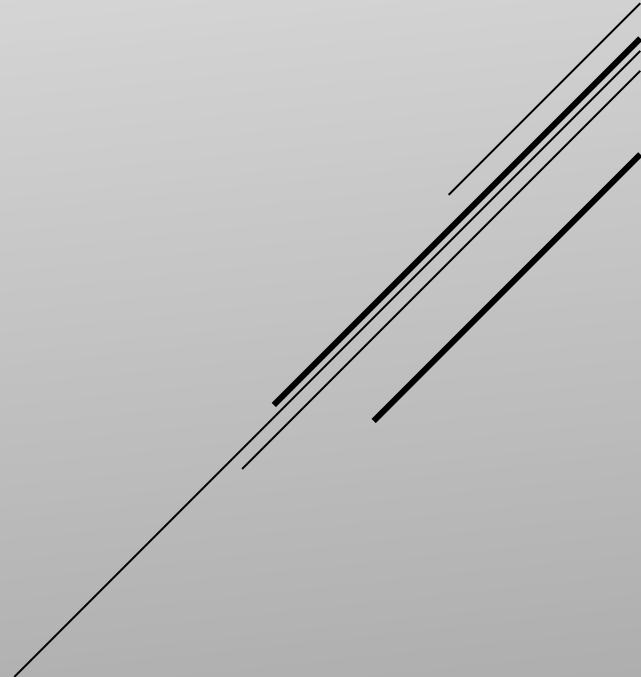


4.4. PROCESAMIENTO DE IMÁGENES UTILIZANDO RESNET50V2

- ▶ Al ser un modelo pre-entrenado tan solo es necesario volver a entrenar las últimas capas
- ▶ 80% de las imágenes para validación y test
- ▶ 20% de las imágenes para entrenamiento

RESULTADOS

CAPÍTULO 5



- 
- ▶ Primeras pruebas: resize 220x220, normalización de los valores de la imagen
 - ▶ Máximo de 50 épocas y creando un checkpoint cada 5

5.1. SIN PREPROCESAMIENTO

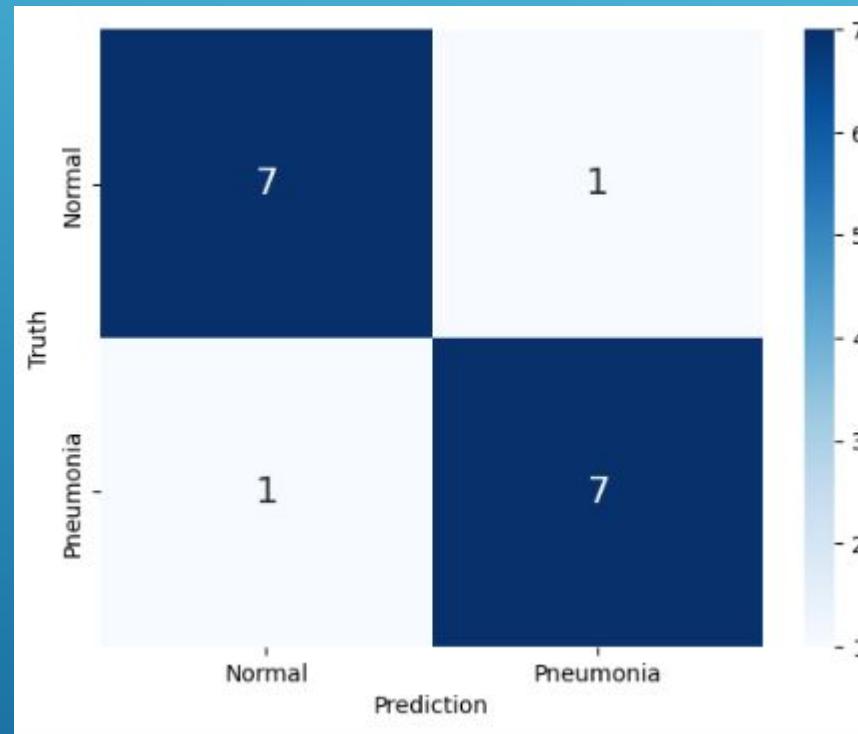


- Buenas métricas iniciales con una precisión del 88% en ambas categorías

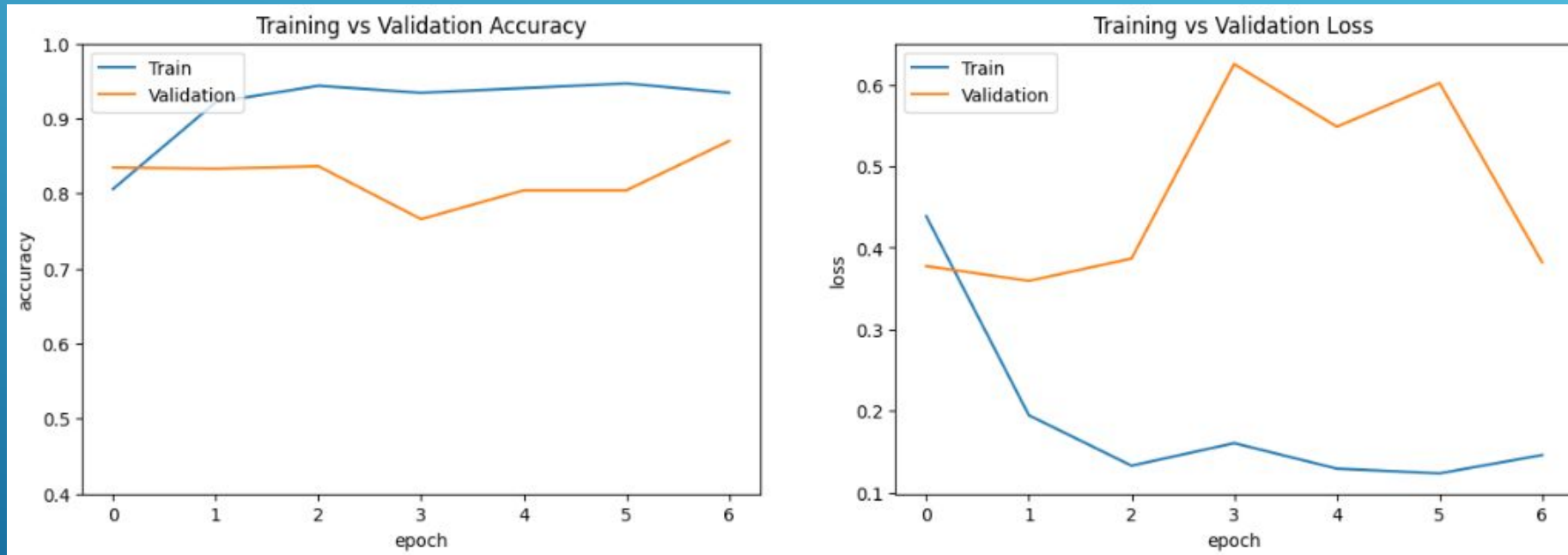
	precision	recall	f1-score	support
0	0.88	0.88	0.88	8
1	0.88	0.88	0.88	8
accuracy			0.88	16
macro avg	0.88	0.88	0.88	16
weighted avg	0.88	0.88	0.88	16

- ▶ En la validación se obtuvo que el modelo acierta 14 imágenes de cada 16
- ▶ En ambas clases hay un fallo por lo que no hay sesgo hacia ningún tipo en particular

	Predicton	Truth	Results
0	0	0	TN
1	0	0	TN
2	0	0	TN
3	0	0	TN
4	0	0	TN
5	1	0	FP
6	0	0	TN
7	0	0	TN
8	1	1	TP
9	1	1	TP
10	1	1	TP
11	1	1	TP
12	0	1	FN
13	1	1	TP
14	1	1	TP
15	1	1	TP



- Métricas sobre el proceso de entrenamiento y validación del modelo

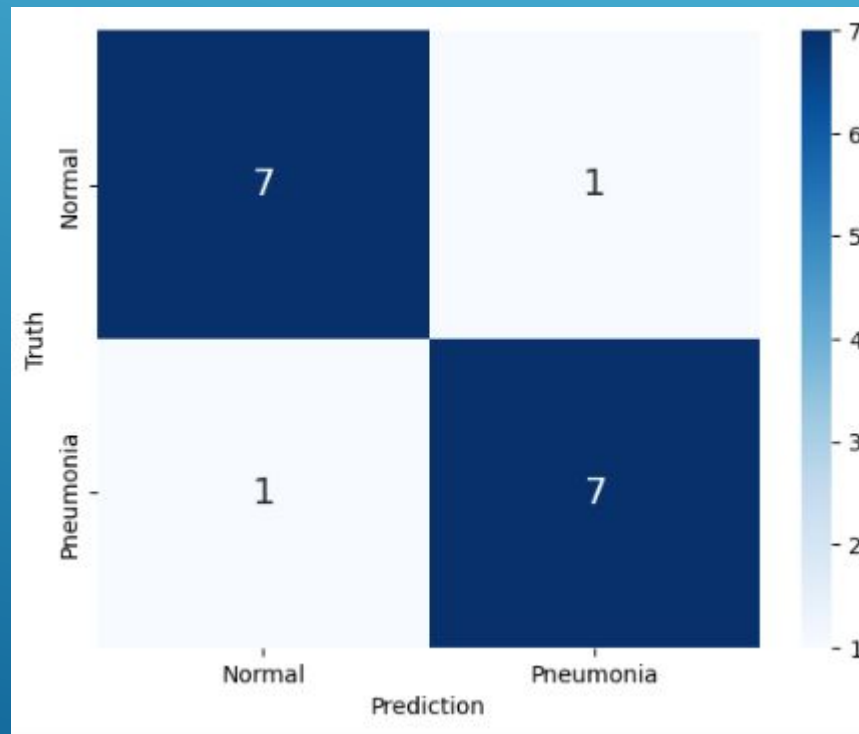


5.2. PRIMER PREPROCESAMIENTO

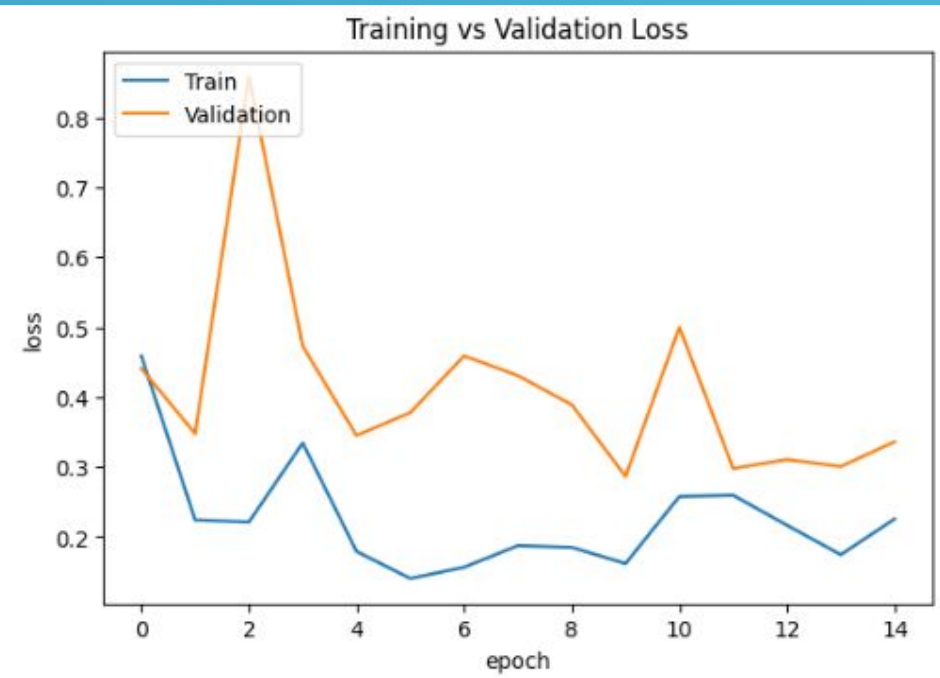
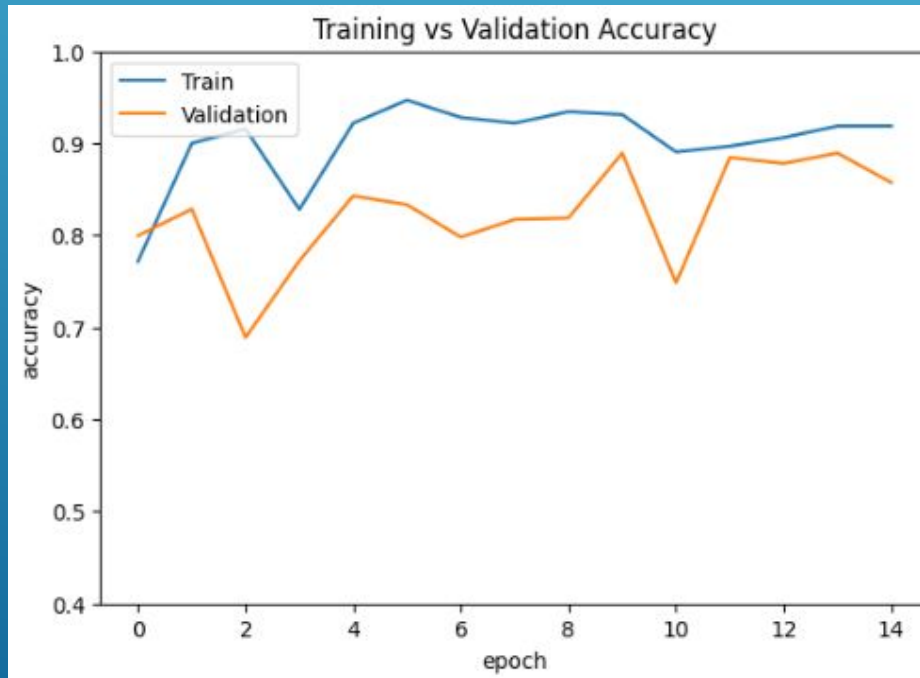
	precision	recall	f1-score	support
0	0.88	0.88	0.88	8
1	0.88	0.88	0.88	8
accuracy			0.88	16
macro avg	0.88	0.88	0.88	16
weighted avg	0.88	0.88	0.88	16

- Métricas prácticamente idénticas al primer entrenamiento

- ▶ Al igual que hemos visto en las métricas anteriores el primer preprocesado no ha tenido mucho impacto en el modelo
- ▶ Mismos aciertos y fallos



- Encontramos una leve diferencia en el proceso de entrenamiento, sin estancamiento y con mayores bajadas del 'validation loss'



```
train_datagen = ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=40,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    zoom_range=0.2,  
    horizontal_flip=True,  
    fill_mode='nearest',  
    preprocessing_function=custom_augmentation  
)
```

5.3. PREPROCESAMIENTO CON AUMENTO DE DATOS

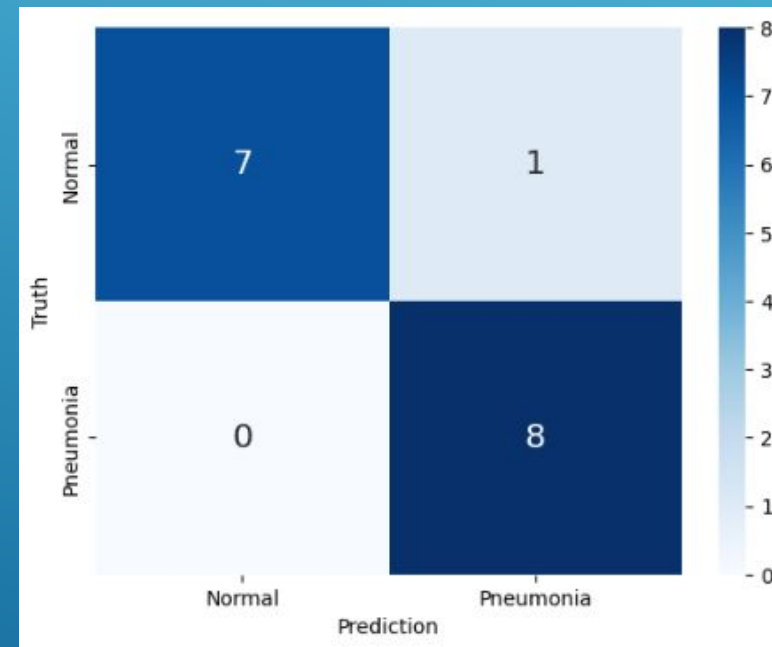
- ▶ En esta tercera etapa hemos utilizado 'data augmentation'
- ▶ Leve filtro de ruido gaussiano
- ▶ Además de otras modificaciones

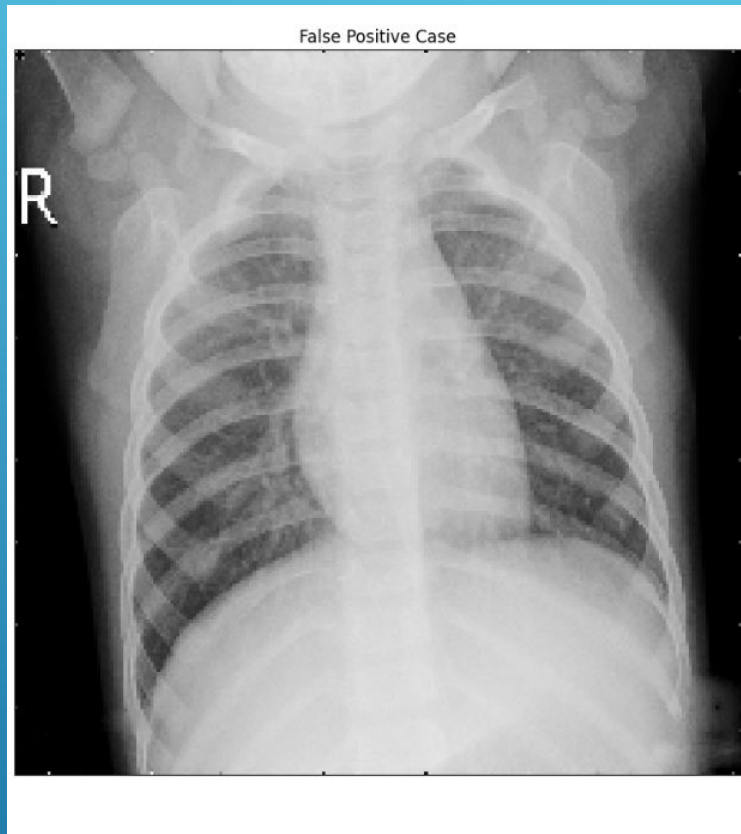
- ▶ En este caso los resultados han mejorado
- ▶ Alcanzamos una precisión total de 94%

	precision	recall	f1-score	support
0	1.00	0.88	0.93	8
1	0.89	1.00	0.94	8
accuracy			0.94	16
macro avg	0.94	0.94	0.94	16
weighted avg	0.94	0.94	0.94	16

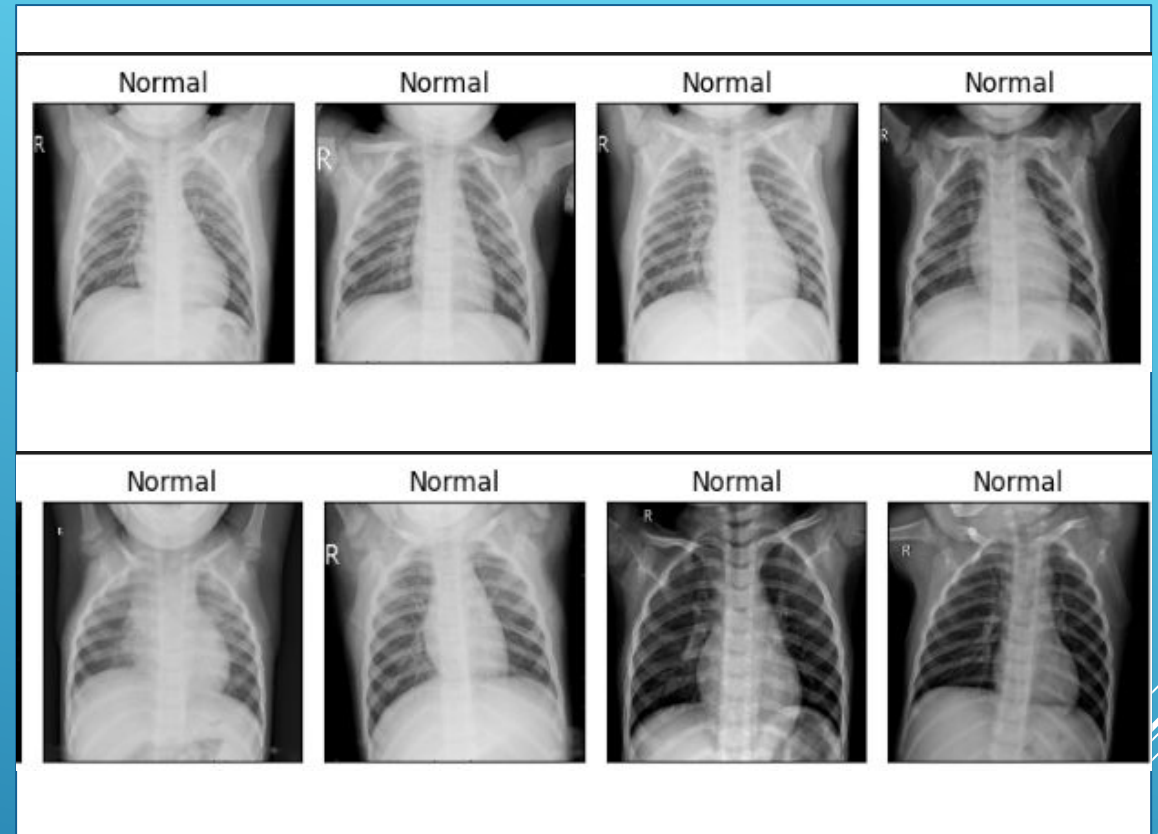
- ▶ Los resultados en la validación fueron de 15 aciertos de las 16 imágenes analizadas
- ▶ Solo un fallo en la clasificación de las imágenes normales

	Predicton	Truth	Results
0	0	0	TN
1	0	0	TN
2	0	0	TN
3	0	0	TN
4	0	0	TN
5	1	0	FP
6	0	0	TN
7	0	0	TN
8	1	1	TP
9	1	1	TP
10	1	1	TP
11	1	1	TP
12	1	1	TP
13	1	1	TP
14	1	1	TP
15	1	1	TP



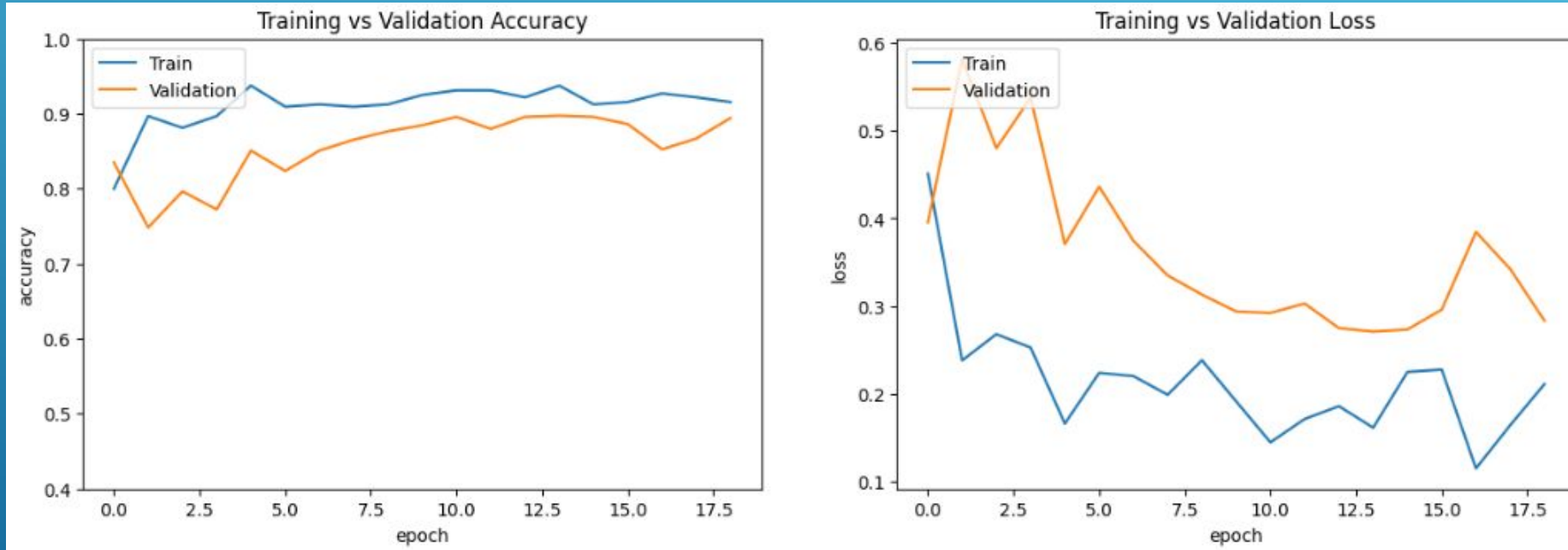


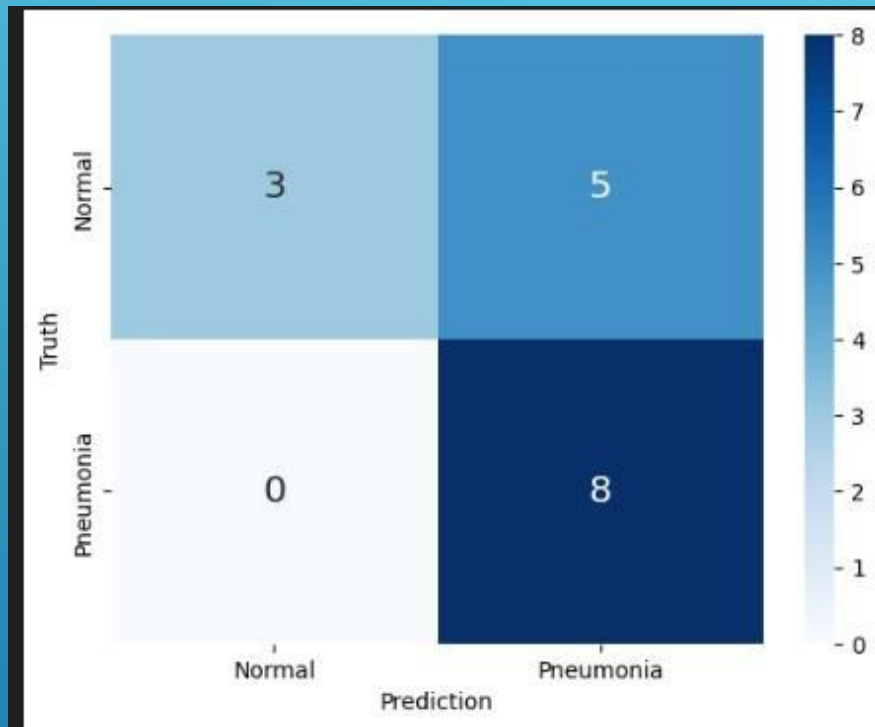
- ▶ Falso positivo en las imágenes normales



- ▶ Ejemplos de imágenes Normales bien clasificadas

- ▶ Respecto a la validación y el entrenamiento hay una disminución de la tendencia del 'validation loss'



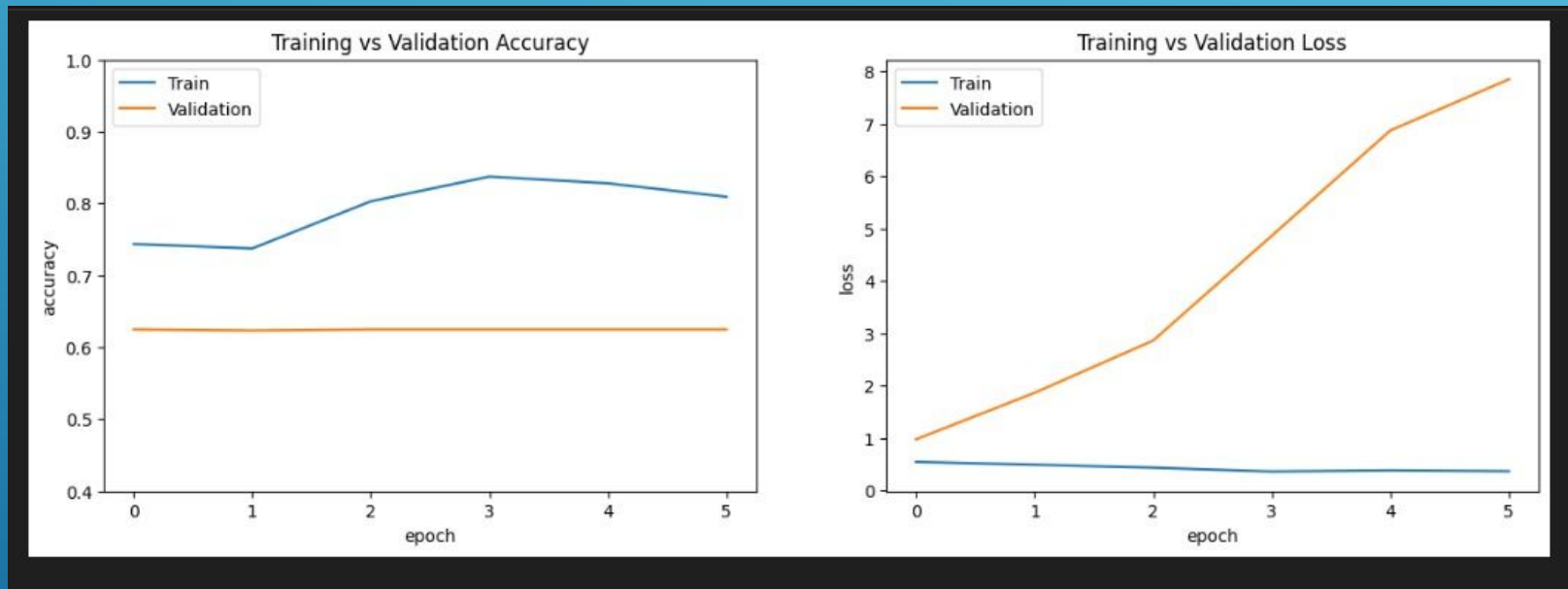


	precision	recall	f1-score	support
0	1.00	0.38	0.55	8
1	0.62	1.00	0.76	8
accuracy			0.69	16
macro avg	0.81	0.69	0.65	16
weighted avg	0.81	0.69	0.65	16

5.4. MALOS RESULTADOS

- ▶ Durante el entrenamiento el uso de un umbral o 'threshold' en el preprocesamiento suponía un sesgo hacia los falsos positivos
- ▶ Se puede observar en los resultados de la matriz de confusión
- ▶ Así mismo vemos como bajan las métricas de precisión, recall y f1

- Por último podemos ver como crece el 'validation loss' durante el proceso de entrenamiento'



CONCLUSIONES Y TRABAJO FUTURO

CAPÍTULO 6

